



資料結構

Data Structure

Lab (加分題)

姓名: 盧敬恩
學號: 114310123

Lab_bonus-Ex1. (heaters)

完成 leetcode-Heaters

Code

```
class Solution {
public:
    int findRadius(vector<int>& houses, vector<int>& heaters) {
        vector<int> hk;
        int max;
        int r;
        int a = houses.size(); //設定a為房子數量
        int b = heaters.size(); //設定b為加熱器數量
        for(int x = 0 ; x<a ;x++) { //巢狀迴圈 內層是房子到每一個加熱器的最大距離
            r= abs(houses[x]-heaters[0]);
            for(int y=0 ;y<b ;y++){
                if(r>abs(houses[x]-heaters[y])){
                    r = abs(houses[x]-heaters[y]);
                }
            }
            hk.push_back(r); //總共有a個元素在hk陣列=房子數量
        }
        max = hk[0];
        for(int z = 0 ; z<hk.size();z++) { //這裡是取最大的r
            if(hk[z]>max){
                max = hk[z];
            }
        }
        return max; //回傳最大的r
    }
}
```

Result

The screenshot displays the LeetCode interface for the 'Heaters' problem. The submission is marked as 'Accepted' with 30/30 test cases passed. Performance metrics show a runtime of 1548 ms (Beats 5.80%) and memory usage of 30.22 MB (Beats 5.12%). A bar chart illustrates the runtime distribution across various test cases. The code is written in C++ and includes detailed comments in Chinese explaining the logic. The test result section shows three cases, all of which were passed.

Accepted 30 / 30 testcases passed
t114310123 submitted at May 27, 2026 21:43

Runtime 1548 ms | Beats 5.80%
Memory 30.22 MB | Beats 5.12%

Code | C++

```
1 class Solution {
2 public:
3     int findRadius(vector<int>& houses, vector<int>& heaters) {
4         vector<int> hk;
5         int max;
6         int r;
7         int a = houses.size(); //設定a為房子數量
8         int b = heaters.size(); //設定b為加熱器數量
9         for(int x = 0 ; x<a ;x++) { //巢狀迴圈 內層是房子到每一個加熱器的最大距離
10             r= abs(houses[x]-heaters[0]);
11             for(int y=0 ;y<b ;y++){
12                 if(r>abs(houses[x]-heaters[y])){
13                     r = abs(houses[x]-heaters[y]);
14                 }
15             }
16             hk.push_back(r); //總共有a個元素在hk陣列=房子數量
17         }
18         max = hk[0];
19         for(int z = 0 ; z<hk.size();z++) { //這裡是取最大的r
20             if(hk[z]>max){
21                 max = hk[z];
22             }
23         }
24         return max; //回傳最大的r
25     }
26 }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

houses = [1,2,3]

heaters = [2]

leetcode.com/problems/heaters/

Accepted 30 / 30 testcases passed
1114310123 submitted at May 27, 2026 21:43

Runtime: 1548 ms | Beat: 5.80%
Memory: 30.22 MB | Beat: 5.12%

0.25% of solutions used 271 ms of runtime

Code | C++

```
1 class Solution {
2 public:
3     int findRadius(vector<int>& houses, vector<int>& heaters) {
4         vector<int> hk;
5         int max;
6         int a = houses.size(); // 設定 a 為房子數量
7         int b = heaters.size(); // 設定 b 為加熱器數量
8         for(int x = 0; x < a; x++) { // 遍歷房子到每一個加熱器的最大距離
9             r = abs(houses[x] - heaters[0]);
10            for(int y = 0; y < b; y++) {
11                if(r > abs(houses[x] - heaters[y])) {
12                    r = abs(houses[x] - heaters[y]);
13                }
14            }
15            hk.push_back(r); // 總共有 a 個元素在 hk 陣列-房子數量
16        }
17        max = hk[0];
18        for(int z = 0; z < hk.size(); z++) { // 這裡是取最大的 r
19            if(hk[z] > max) {
20                max = hk[z];
21            }
22        }
23        return max; // 回傳最大的 r
24    }
25 }
26 }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

houses = [1,2,3,4]

heaters =

leetcode.com/problems/heaters/

Accepted 30 / 30 testcases passed
1114310123 submitted at May 27, 2026 21:43

Runtime: 1548 ms | Beat: 5.80%
Memory: 30.22 MB | Beat: 5.12%

0.19% of solutions used 336 ms of runtime

Code | C++

```
1 class Solution {
2 public:
3     int findRadius(vector<int>& houses, vector<int>& heaters) {
4         vector<int> hk;
5         int max;
6         int r;
7         int a = houses.size();
8         int b = heaters.size();
9         for(int x = 0; x < a; x++) { // 設定 a 為房子數量
10            for(int y = 0; y < b; y++) { // 設定 b 為加熱器數量
11                r = abs(houses[x] - heaters[y]);
12                if(r > abs(houses[x] - heaters[0])) {
13                    r = abs(houses[x] - heaters[y]);
14                }
15            }
16            hk.push_back(r); // 總共有 a 個元素在 hk 陣列-房子數量
17        }
18        max = hk[0];
19        for(int z = 0; z < hk.size(); z++) { // 這裡是取最大的 r
20            if(hk[z] > max) {
21                max = hk[z];
22            }
23        }
24        return max; // 回傳最大的 r
25    }
26 }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

houses = [1,5]

heaters =

Discussion

這次的加分題必較簡單,我沒有用半點ai很有成就感,
但是我使用了巢狀迴圈暴力破解法讓我的時間複雜度變成 $O(n^2)$
這很不好,感覺可以透過房子座標往左和右邊推直到遇到heater之類的思維來優化
但是我有點累所以就用暴力破解必較

$x = \text{houses point}$

$y = \text{heaters - point}$



